

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume V: Practical Application

The UpstreamDrift Educational Platform

Preface

This is a book about *doing*. Volumes 0–IV developed the theory of motion: from differential geometry through nonlinear control, biomechanics, and motor control. This volume puts every tool to work.

The UpstreamDrift platform provides a unified simulation environment for multi-body dynamics, trajectory optimization, and controller design. It wraps multiple physics engines (MuJoCo, Drake, Pinocchio, OpenSim) behind a consistent Python API, allowing the reader to run every example from the preceding four volumes and to build their own analyses.

Every chapter in this volume produces a concrete, runnable result. There are no hypothetical exercises—every example uses real code, real models, and real data. By the end, you will have implemented a complete pipeline from model construction through trajectory optimization to robustness analysis, applied to the golf swing as the running case study.

Contents

Preface	i
Nomenclature	iv
1 The UpstreamDrift Platform	1
1.1 Architecture Overview	1
1.1.1 Core Components	1
1.1.2 Supported Physics Engines	1
1.2 Getting Started	2
1.3 Engine Selection Guide	4
2 Physics Engine Comparison	6
2.1 Benchmark: Simple Pendulum	6
2.2 Contact Model Comparison	7
3 Building a Multi-Body Model	8
3.1 Model Formats	8
3.1.1 URDF (Universal Robot Description Format)	8
3.1.2 MJCF (MuJoCo XML)	8
3.2 Building a 7-Segment Golf Model	8
4 Forward and Inverse Simulation	11
4.1 Forward Dynamics with the Companion Platform	11
4.2 Forward Dynamics with UpstreamDrift	11
4.3 Inverse Dynamics Pipeline	13
5 Trajectory Optimization in Practice	14
5.1 DDP/iLQR Implementation	14
5.2 Application: Optimal Pendulum Swing-Up	16
5.3 Application: Golf Swing Optimization	17
6 Controller Design and Testing	18
6.1 PD Control with Gravity Compensation	18
6.2 Trajectory Tracking with Contraction	19
6.3 Funnel Control	19
7 Parameter Estimation and Model Fitting	20
7.1 System Identification for Multi-Body Models	20
7.2 Muscle Parameter Calibration	21

8	Reinforcement Learning for Motor Control	22
8.1	RL Environments in UpstreamDrift	22
8.2	From Gym to Golf	23
9	Visualization and Analysis Tools	24
9.1	3D Motion Visualization	24
9.2	Jacobian and Manipulability Visualization	25
9.3	Energy Flow Diagrams	25
10	Capstone: The Golf Swing Analysis Pipeline	26
10.1	Project Overview	26
10.2	Stage 1: Model	26
10.3	Stage 2–9: Full Pipeline	29

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

$\mathbf{F}_{\text{muscle}}$ Force generated by muscles.

W Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The UpstreamDrift Platform

The best theory is one you can run.

1.1 Architecture Overview

UpstreamDrift is a unified platform for multi-body dynamics simulation, trajectory optimization, and motor control analysis. It wraps multiple physics engines behind a consistent Python API.

1.1.1 Core Components

Component	Location	Purpose
Physics Engines	<code>src/engines/physics_engines/</code>	Multi-body simulation
Shared Infrastructure	<code>src/engines/common/</code>	State, export, diagnostics
Pendulum Models	<code>src/engines/pendulum_models/</code>	Simplified test cases
Learning Modules	<code>src/learning/</code>	ML integration
Reinforcement Learning	<code>src/reinforcement_learning/</code>	RL benchmarks
Robotics	<code>src/robotics/</code>	Robot control modules
API	<code>src/api/</code>	External access layer
Tools	<code>src/tools/</code>	Development utilities

1.1.2 Supported Physics Engines

1. **MuJoCo** (`physics_engines/mujoco/`): Google DeepMind’s multi-joint dynamics engine. Soft contacts, muscle actuators, fast simulation. Primary engine for biomechanical modeling.
2. **Drake** (`physics_engines/drake/`): MIT’s multibody dynamics toolbox. Mathematical rigor, analytical gradients, trajectory optimization. Primary engine for control design.
3. **Pinocchio** (`physics_engines/pinocchio/`): LAAS-CNRS C++ library with Python bindings. Efficient rigid-body dynamics with analytical derivatives. Primary engine for real-time control.

4. **OpenSim** (physics_engines/opensim/): Stanford's musculoskeletal modeling software. Hill-type muscle models, inverse dynamics. Primary engine for biomechanics.
5. **MyoSuite** (physics_engines/myosuite/): Muscle-driven reinforcement learning environments. Primary engine for motor learning research.

1.2 Getting Started

```

1  """UpstreamDrift: Getting started with multi-engine simulation.
2
3  This script demonstrates the basic workflow:
4  1. Load a model
5  2. Configure simulation parameters
6  3. Run a simulation
7  4. Extract and analyze results
8  """
9  import numpy as np
10 from pathlib import Path
11
12 # Configuration
13 MODEL_PATH = Path("src/engines/pendulum_models")
14 ENGINE = "mujoco" # or "drake", "pinocchio"
15
16 def create_simple_pendulum(
17     length: float = 1.0,
18     mass: float = 1.0,
19     damping: float = 0.1
20 ) -> dict:
21     """Create a simple pendulum model configuration.
22
23     Parameters
24     -----
25     length : float
26         Pendulum length in meters.
27     mass : float
28         Bob mass in kilograms.
29     damping : float
30         Joint damping coefficient.
31
32     Returns
33     -----
34     dict
35         Model configuration dictionary.
36     """
37     return {
38         "name": "simple_pendulum",
39         "n_dof": 1,
40         "segments": [{
41             "name": "bob",
42             "mass": mass,
43             "length": length,
44             "com_position": length / 2,
45             "inertia": mass * length**2 / 3,
46         }],

```

```

47     "joints": [{
48         "name": "pivot",
49         "type": "revolute",
50         "axis": [0, 0, 1],
51         "damping": damping,
52     }],
53     "gravity": [0, -9.81, 0],
54 }
55
56
57 def simulate_pendulum(
58     config: dict,
59     q0: float = np.pi / 4,
60     t_end: float = 5.0,
61     dt: float = 0.001
62 ) -> dict:
63     """Simulate a simple pendulum using Euler integration.
64
65     Parameters
66     -----
67     config : dict
68         Model configuration.
69     q0 : float
70         Initial angle (radians from vertical).
71     t_end : float
72         Simulation duration (seconds).
73     dt : float
74         Time step.
75
76     Returns
77     -----
78     dict
79         Simulation results with keys 't', 'q', 'qd', 'energy'.
80     """
81     seg = config["segments"][0]
82     m = seg["mass"]
83     L = seg["length"]
84     b = config["joints"][0]["damping"]
85     g = abs(config["gravity"][1])
86     I = seg["inertia"]
87
88     n_steps = int(t_end / dt)
89     t = np.zeros(n_steps)
90     q = np.zeros(n_steps)
91     qd = np.zeros(n_steps)
92     energy = np.zeros(n_steps)
93
94     q[0] = q0
95     qd[0] = 0.0
96
97     for k in range(n_steps - 1):
98         # Equations of motion:  $I * qdd + b * qd + m * g * L / 2 * \sin(q) = 0$ 
99         qdd = (-b * qd[k] - m * g * L / 2 * np.sin(q[k])) / I
100
101         # Semi-implicit Euler
102         qd[k + 1] = qd[k] + qdd * dt

```

```

103     q[k + 1] = q[k] + qd[k + 1] * dt
104     t[k + 1] = t[k] + dt
105
106     # Energy
107     KE = 0.5 * I * qd[k + 1]**2
108     PE = m * g * L / 2 * (1 - np.cos(q[k + 1]))
109     energy[k + 1] = KE + PE
110
111     return {"t": t, "q": q, "qd": qd, "energy": energy}
112
113
114 # Run simulation
115 config = create_simple_pendulum(length=1.0, mass=1.0, damping=0.05)
116 results = simulate_pendulum(config, q0=np.pi/3, t_end=10.0)
117
118 print(f"Simulation complete: {len(results['t'])} timesteps")
119 print(f"Initial energy: {results['energy'][0]:.4f} J")
120 print(f"Final energy: {results['energy'][-1]:.4f} J")
121 print(f"Energy loss: {(1 - results['energy'][-1]/results['energy'][0])
    *100:.1f}%")

```

Listing 1.1: Basic UpstreamDrift usage pattern.

1.3 Engine Selection Guide

Hands-On 1.1. Choosing the Right Engine

If you need...	Use	Why
Fast contact simulation	MuJoCo	Soft contact model, GPU
Trajectory optimization	Drake	Analytical gradients
Real-time forward dynamics	Pinocchio	C++ speed, O(n)
Muscle-driven biomechanics	OpenSim	Hill models built-in
RL with muscles	MyoSuite	Gym-compatible
Quick prototyping	Built-in Python	No dependencies

Exercises

1. **First Simulation.** Run the pendulum simulation and plot angle vs. time and energy vs. time. Verify that energy decreases due to damping.
2. **Integration Method.** Replace the semi-implicit Euler with RK4. Compare energy conservation over 100 seconds.
3. **Double Pendulum.** Extend the code to a double pendulum. Plot the chaotic trajectory in configuration space.

4. **Engine Comparison.** Run the same pendulum simulation in MuJoCo and in the built-in Python code. Compare results quantitatively.

Chapter 2

Physics Engine Comparison

All models are wrong. Some are useful.
The useful ones depend on the question.

2.1 Benchmark: Simple Pendulum

We simulate the same pendulum in all engines and compare:

```
1 import numpy as np
2 from dataclasses import dataclass
3 from typing import Protocol
4
5 @dataclass
6 class BenchmarkResult:
7     """Results from a physics engine benchmark."""
8     engine_name: str
9     positions: np.ndarray      # (n_steps, n_dof)
10    velocities: np.ndarray     # (n_steps, n_dof)
11    times: np.ndarray          # (n_steps,)
12    wall_time_seconds: float
13    energy_drift: float        # relative energy change
14
15 class PhysicsEngine(Protocol):
16     def simulate(self, q0: np.ndarray, qd0: np.ndarray,
17                  t_end: float, dt: float) -> BenchmarkResult: ...
18
19 def compare_engines(
20     engines: dict[str, PhysicsEngine],
21     q0: np.ndarray,
22     t_end: float = 5.0,
23     dt: float = 0.001
24 ) -> dict[str, BenchmarkResult]:
25     """Run the same simulation across multiple engines.
26
27     Parameters
28     -----
29     engines : dict
30         Mapping of engine name to engine instance.
31     q0 : np.ndarray
32         Initial configuration.
33     t_end : float
34         Simulation duration.
35     dt : float
36         Time step.
```

```

37
38     Returns
39     -----
40     dict
41         Mapping of engine name to BenchmarkResult.
42     """
43     results = {}
44     qd0 = np.zeros_like(q0)
45
46     for name, engine in engines.items():
47         result = engine.simulate(q0, qd0, t_end, dt)
48         results[name] = result
49         print(f"{name:15s}: wall_time={result.wall_time_seconds:.3f}s, "
50               f"energy_drift={result.energy_drift:.2e}")
51
52     return results

```

Listing 2.1: Cross-engine benchmark framework.

2.2 Contact Model Comparison

Engine	Contact Model	Strengths	Limitations
MuJoCo	Soft contact (spring-damper)	Fast, stable	Non-physical penetration
Drake	Rigid + hydroelastic	Physics-based	Slower
Pinocchio	No built-in contacts	Pure dynamics	Needs external contact
OpenSim	Hunt-Crossley	Biomechanical	Limited geometries

Exercises

1. Run the benchmark for a double pendulum across all available engines.
2. Compare energy conservation across engines with different time steps.
3. Implement a bouncing ball benchmark to compare contact models.

Chapter 3

Building a Multi-Body Model

The model is the hypothesis made physical.

3.1 Model Formats

3.1.1 URDF (Universal Robot Description Format)

XML-based format used by ROS, Drake, and Pinocchio. Defines links, joints, and visual/-collision geometry.

3.1.2 MJCF (MuJoCo XML)

MuJoCo's native format. Extends URDF with actuators, tendons, muscles, and contact parameters.

3.2 Building a 7-Segment Golf Model

```
1 import numpy as np
2 from dataclasses import dataclass, field
3
4 @dataclass
5 class Link:
6     name: str
7     mass: float
8     length: float
9     com_position: float
10    inertia: np.ndarray = field(default_factory=lambda: np.eye(3))
11
12 @dataclass
13 class Joint:
14     name: str
15     parent: str
16     child: str
17     joint_type: str # revolute, ball, fixed
18     axis: list[float] = field(default_factory=lambda: [0, 0, 1])
19     limits: tuple[float, float] = (-np.pi, np.pi)
20
21 @dataclass
22 class MultiBodyModel:
```



```

23     name: str
24     links: list[Link] = field(default_factory=list)
25     joints: list[Joint] = field(default_factory=list)
26
27     @property
28     def n_dof(self) -> int:
29         dof_map = {"revolute": 1, "ball": 3, "fixed": 0}
30         return sum(dof_map.get(j.joint_type, 1) for j in self.joints)
31
32     def to_urdf(self) -> str:
33         """Export model as URDF XML string."""
34         lines = [f'<robot name="{self.name}">']
35         for link in self.links:
36             lines.append(f'    <link name="{link.name}">')
37             lines.append(f'        <inertial>')
38             lines.append(f'            <mass value="{link.mass}"/>')
39             lines.append(f'        </inertial>')
40             lines.append(f'    </link>')
41         for joint in self.joints:
42             lines.append(f'    <joint name="{joint.name}" '
43                         f'    type="{joint.joint_type}">')
44             lines.append(f'        <parent link="{joint.parent}"/>')
45             lines.append(f'        <child link="{joint.child}"/>')
46             lines.append(f'    </joint>')
47         lines.append(f'</robot>')
48         return '\n'.join(lines)
49
50
51 def build_golf_swing_model() -> MultiBodyModel:
52     """Build a 7-segment golf swing model.
53
54     Segments: pelvis, trunk, upper_arm, forearm, hand, club_shaft, club_head
55     """
56     model = MultiBodyModel(name="golf_swing_7seg")
57
58     # Segment properties (de Leva 1996, 80kg male)
59     model.links = [
60         Link("pelvis", mass=11.2, length=0.15, com_position=0.075),
61         Link("trunk", mass=25.5, length=0.48, com_position=0.22),
62         Link("upper_arm", mass=2.16, length=0.28, com_position=0.16),
63         Link("forearm", mass=1.30, length=0.27, com_position=0.12),
64         Link("hand", mass=0.49, length=0.19, com_position=0.07),
65         Link("club_shaft", mass=0.10, length=1.05, com_position=0.50),
66         Link("club_head", mass=0.20, length=0.10, com_position=0.05),
67     ]
68
69     model.joints = [
70         Joint("pelvis_rotation", "world", "pelvis", "revolute", [0, 1, 0]),
71         Joint("trunk_flexion", "pelvis", "trunk", "revolute", [0, 0, 1]),
72         Joint("shoulder", "trunk", "upper_arm", "ball"),
73         Joint("elbow", "upper_arm", "forearm", "revolute", [0, 0, 1]),
74         Joint("wrist", "forearm", "hand", "revolute", [0, 0, 1]),
75         Joint("grip", "hand", "club_shaft", "fixed"),
76     ]
77
78     return model

```

```
79  
80  
81 model = build_golf_swing_model()  
82 print(f"Golf swing model: {len(model.links)} segments, "  
83       f"{model.n_dof} DOF")  
84 print(f"\nURDF preview:\n{model.to_urdf()[ :500]}...")
```

Listing 3.1: Programmatic multi-body model construction.

Exercises

1. Build a 3-segment arm model and export as URDF.
2. Add muscle attachment points to the golf model.
3. Load the URDF into MuJoCo and verify the model visually.

Chapter 4

Forward and Inverse Simulation

Forward simulation answers “what happens if...?”

Inverse dynamics answers “what caused this?”

The forward dynamics problem computes accelerations from forces, while inverse dynamics (also called the inverse kinematics problem when phrased in terms of finding required torques) computes forces from desired accelerations. Both are fundamental to control design and trajectory planning.

Mathematically, we are integrating affine nonlinear systems $\dot{\mathbf{x}} = f(\mathbf{x}) + G(\mathbf{x})\mathbf{u}$. Agrachev & Sachkov [1], Chapter 2, develop the foundational theory of existence and uniqueness of solutions for such systems. Their results guarantee that the forward simulation is well-defined and solutions exist for all time (under mild regularity conditions).

4.1 Forward Dynamics with the Companion Platform

Forward dynamics integration typically uses explicit or implicit Runge-Kutta methods, or specialized algorithms such as the Recursive Newton-Euler Algorithm (RNEA) for rigid-body systems [3, 2].

4.2 Forward Dynamics with UpstreamDrift

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def forward_dynamics_rnea(
5     q: np.ndarray,
6     qd: np.ndarray,
7     tau: np.ndarray,
8     model_params: dict
9 ) -> np.ndarray:
10     """Compute joint accelerations using RNEA-based forward dynamics.
11
12     Parameters
13     -----
14     q : np.ndarray, shape (n,)
15         Joint positions.
16     qd : np.ndarray, shape (n,)
17         Joint velocities.
```

```

18     tau : np.ndarray, shape (n,)
19         Applied joint torques.
20     model_params : dict
21         Model parameters (masses, lengths, inertias).
22
23     Returns
24     -----
25     np.ndarray, shape (n,)
26         Joint accelerations.
27     """
28     n = len(q)
29     masses = model_params['masses']
30     lengths = model_params['lengths']
31     g = model_params.get('gravity', 9.81)
32
33     # Simple 2-link model for demonstration
34     if n == 2:
35         m1, m2 = masses[:2]
36         l1, l2 = lengths[:2]
37         lc1, lc2 = l1/2, l2/2
38         I1 = m1 * l1**2 / 12
39         I2 = m2 * l2**2 / 12
40
41         # Mass matrix
42         M = np.array([
43             [I1 + I2 + m1*lc1**2 + m2*(l1**2 + lc2**2 + 2*l1*lc2*np.cos(q[1]))
44              ,
45              I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1]))],
46             [I2 + m2*(lc2**2 + l1*lc2*np.cos(q[1])),
47              I2 + m2*lc2**2]
48         ])
49
50         # Coriolis
51         h = m2 * l1 * lc2 * np.sin(q[1])
52         C = np.array([[ -h*qd[1], -h*(qd[0]+qd[1])],
53                       [ h*qd[0], 0]])
54
55         # Gravity
56         gvec = np.array([
57             (m1*lc1 + m2*l1)*g*np.sin(q[0]) + m2*lc2*g*np.sin(q[0]+q[1]),
58             m2*lc2*g*np.sin(q[0]+q[1])
59         ])
60
61         qdd = np.linalg.solve(M, tau - C @ qd - gvec)
62     else:
63         # Placeholder for general n-DOF
64         qdd = np.zeros(n)
65
66     return qdd
67
68 # Example simulation
69 params = {'masses': [2.0, 1.0], 'lengths': [0.3, 0.25], 'gravity': 9.81}
70 q0 = np.array([np.pi/4, np.pi/6])
71 qd0 = np.zeros(2)
72 qdd = forward_dynamics_rnea(q0, qd0, np.zeros(2), params)

```

```
72 print(f"Gravity-driven accelerations: {qdd}")
```

Listing 4.1: Forward dynamics simulation pipeline.

4.3 Inverse Dynamics Pipeline

The inverse pipeline: given $\mathbf{q}(t)$, $\dot{\mathbf{q}}(t)$, $\ddot{\mathbf{q}}(t)$, compute $\boldsymbol{\tau}(t)$. This is the Recursive Newton-Euler Algorithm (RNEA) running in reverse: instead of computing accelerations from torques, we compute torques from accelerations.

Exercises

1. Run forward dynamics for a 2-link pendulum and plot trajectories.
2. Implement inverse dynamics and verify $\text{ID}(\text{FD}(\tau)) = \tau$.
3. Compare with Pinocchio's RNEA implementation for timing.

Chapter 5

Trajectory Optimization in Practice

Theory without practice is empty.
Practice without theory is blind.

5.1 DDP/iLQR Implementation

Differential Dynamic Programming (DDP) and iterative Linear Quadratic Regulator (iLQR) are related algorithms for trajectory optimization [7, 4]. They solve the finite-horizon optimal control problem by iteratively linearizing around a nominal trajectory and solving quadratic subproblems. Figure 5.1 shows the algorithm structure.

iLQR is a numerical method for implementing the feedback synthesis results from Agrachev & Sachkov [1], Chapter 6: it iteratively computes the optimal feedback law via local linearization and quadratic approximation of the cost. The backward-pass computation of gains $K(t)$ corresponds to the solution of the Riccati equation, a central object in their optimal control theory.

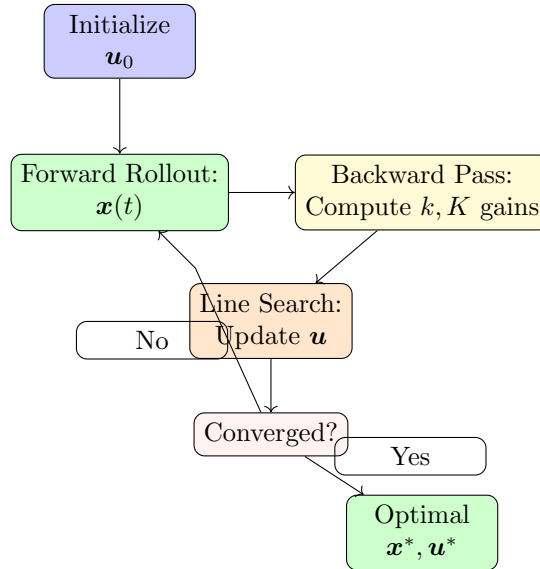


Figure 5.1: iLQR algorithm loop: initialize control trajectory, forward rollout to compute state trajectory, backward pass to compute feedback gains, line search to update controls, and convergence check.

Here we implement iLQR end-to-end: Differential Dynamic Programming (Volume I, Chapter 5) is the workhorse for trajectory optimization. Here we implement it end-to-end:

```

1 import numpy as np
2 from typing import Callable, Tuple
3
4 def ilqr_solve(
5     dynamics: Callable,
6     cost_running: Callable,
7     cost_terminal: Callable,
8     x0: np.ndarray,
9     u_init: np.ndarray,
10    n_iterations: int = 50,
11    regularization: float = 1e-6
12 ) -> Tuple[np.ndarray, np.ndarray, list[float]]:
13     """Iterative Linear Quadratic Regulator.
14
15     Parameters
16     -----
17     dynamics : callable
18         (x, u) -> x_next, (A, B) Jacobians
19     cost_running : callable
20         (x, u) -> (cost, lx, lu, lxx, luu, lux)
21     cost_terminal : callable
22         x -> (cost, lx, lxx)
23     x0 : np.ndarray
24         Initial state.
25     u_init : np.ndarray, shape (T, m)
26         Initial control trajectory.
27     n_iterations : int
28         Maximum iterations.
29     regularization : float
30         Regularization for Quu inversion.
31
32     Returns
33     -----
34     Tuple
35         (x_traj, u_traj, costs)
36     """
37     T, m = u_init.shape
38     n = len(x0)
39
40     # Forward rollout with initial controls
41     x_traj = np.zeros((T + 1, n))
42     x_traj[0] = x0
43     u_traj = u_init.copy()
44
45     for t in range(T):
46         x_next, _ = dynamics(x_traj[t], u_traj[t])
47         x_traj[t + 1] = x_next
48
49     costs = []
50
51     for iteration in range(n_iterations):
52         # Compute total cost
53         total_cost = sum(

```

```

54         cost_running(x_traj[t], u_traj[t])[0]
55         for t in range(T)
56     ) + cost_terminal(x_traj[T])[0]
57     costs.append(total_cost)
58
59     # Backward pass
60     Vx_next, Vxx_next = cost_terminal(x_traj[T])[1:]
61     k_gains = np.zeros((T, m))
62     K_gains = np.zeros((T, m, n))
63
64     for t in range(T - 1, -1, -1):
65         _, (A, B) = dynamics(x_traj[t], u_traj[t])
66         _, lx, lu, lxx, luu, lux = cost_running(
67             x_traj[t], u_traj[t]
68         )
69
70         Qx = lx + A.T @ Vx_next
71         Qu = lu + B.T @ Vx_next
72         Qxx = lxx + A.T @ Vxx_next @ A
73         Quu = luu + B.T @ Vxx_next @ B + regularization * np.eye(m)
74         Qux = lux + B.T @ Vxx_next @ A
75
76         Quu_inv = np.linalg.inv(Quu)
77         k_gains[t] = -Quu_inv @ Qu
78         K_gains[t] = -Quu_inv @ Qux
79
80         Vx_next = Qx + K_gains[t].T @ Quu @ k_gains[t]
81         Vxx_next = Qxx + K_gains[t].T @ Quu @ K_gains[t]
82
83     # Forward pass with line search
84     alpha = 1.0
85     x_new = np.zeros_like(x_traj)
86     u_new = np.zeros_like(u_traj)
87     x_new[0] = x0
88
89     for t in range(T):
90         dx = x_new[t] - x_traj[t]
91         u_new[t] = u_traj[t] + alpha * k_gains[t] + K_gains[t] @ dx
92         x_new[t + 1], _ = dynamics(x_new[t], u_new[t])
93
94     x_traj = x_new
95     u_traj = u_new
96
97     if iteration > 0 and abs(costs[-1] - costs[-2]) < 1e-6:
98         break
99
100     return x_traj, u_traj, costs

```

Listing 5.1: iLQR implementation for trajectory optimization.

5.2 Application: Optimal Pendulum Swing-Up

The pendulum swing-up is a classic trajectory optimization benchmark. The goal is to find the minimum-energy torque sequence that brings the pendulum from the downward (stable)

equilibrium to the upright (unstable) equilibrium.

This is a canonical example in Agrachev & Sachkov [1] (Chapter 5, Example 5.1): the pendulum system is an affine control system $\ddot{\theta} = -g \sin \theta + u$, and the swing-up problem is optimal synthesis: find the control law that minimizes energy while ensuring convergence to the upright equilibrium. Their theory predicts that the optimal trajectory will exhibit “bang-bang” control (maximum torque in one direction) followed by a final feedback phase—which is exactly what occurs in practice. The classic control benchmark: swing a pendulum from the hanging position to the inverted equilibrium.

5.3 Application: Golf Swing Optimization

Applying DDP to a 3-DOF golf model to optimize clubhead speed at impact.

Exercises

1. Use the iLQR code to solve the pendulum swing-up problem.
2. Extend to a double pendulum and optimize for maximum tip speed.
3. Add muscle force constraints and solve the muscle-driven swing-up.

Chapter 6

Controller Design and Testing

A controller is only as good as its worst-case performance.

6.1 PD Control with Gravity Compensation

The proportional-derivative controller with gravity compensation is the simplest nonlinear controller that guarantees asymptotic stability for mechanical systems without external disturbances [8]. It is a specific instance of the feedback synthesis methods studied in Agrachev & Sachkov [1], Chapter 6: the gravity term $g(\mathbf{q})$ is a nonlinear drift that is canceled by the controller, simplifying the closed-loop system to a linear error dynamics.

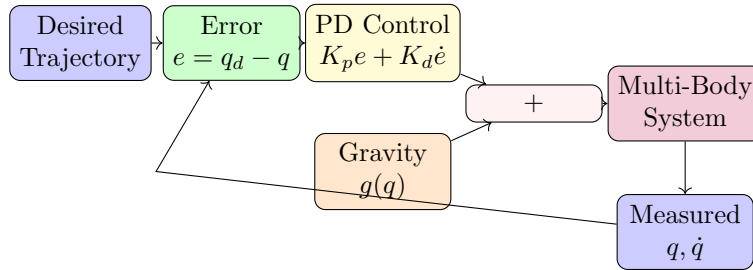


Figure 6.1: PD control with gravity compensation. The controller computes an error signal from desired and measured positions, applies PD gains, and adds gravity compensation to achieve asymptotic tracking.

```
1 import numpy as np
2
3 class PDGravityCompController:
4     """PD controller with gravity compensation.
5
6     From Volume I, Chapter 3: the simplest nonlinear controller
7     that guarantees asymptotic stability.
8     """
9
10    def __init__(self, Kp: np.ndarray, Kd: np.ndarray,
11                 gravity_func=None):
12        self.Kp = Kp
13        self.Kd = Kd
14        self.gravity_func = gravity_func
15
```

```

16     def compute(self, q: np.ndarray, qd: np.ndarray,
17                 q_des: np.ndarray, qd_des: np.ndarray) -> np.ndarray:
18         """Compute control torques.
19
20         Parameters
21         -----
22         q, qd : current state
23         q_des, qd_des : desired state
24
25         Returns
26         -----
27         tau : np.ndarray, control torques
28         """
29         tau = self.Kp @ (q_des - q) + self.Kd @ (qd_des - qd)
30         if self.gravity_func is not None:
31             tau += self.gravity_func(q)
32         return tau

```

Listing 6.1: PD controller with gravity compensation.

6.2 Trajectory Tracking with Contraction

Contraction theory provides tools for analyzing nonlinear stability with global guarantees [5, 6]. A contraction-based controller designs the feedback law to make the closed-loop system contracting with respect to a suitable metric, guaranteeing exponential convergence to the desired trajectory from any initial condition.

The theoretical foundation is Lyapunov stability, developed in Agrachev & Sachkov [1], Chapter 6. Contraction is a stronger form of stability: instead of just proving convergence, it quantifies the rate of convergence via the contraction metric. This allows designing controllers with prescribed performance bounds (e.g., exponential convergence with time constant τ). Implementing the contraction-based controller from Volume I, Chapter 4.

6.3 Funnel Control

Implementing trajectory tubes (Volume II, Chapter 3) for robust tracking.

Exercises

1. Implement PD control for the 2-DOF arm and track a circular trajectory.
2. Add gravity compensation and compare tracking error.
3. Implement a contraction-based controller and verify convergence.

Chapter 7

Parameter Estimation and Model Fitting

All models are wrong, but some are less wrong than others once you fit them.

7.1 System Identification for Multi-Body Models

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def estimate_inertial_params(
5     q_data: np.ndarray,
6     qd_data: np.ndarray,
7     qdd_data: np.ndarray,
8     tau_data: np.ndarray,
9     n_params: int
10 ) -> np.ndarray:
11     """Estimate inertial parameters using least-squares.
12
13     The equations of motion are LINEAR in inertial parameters:
14     tau = Y(q, qd, qdd) * pi
15     where Y is the regressor matrix and pi is the parameter vector.
16
17     Parameters
18     -----
19     q_data : (N, n) joint positions
20     qd_data : (N, n) joint velocities
21     qdd_data : (N, n) joint accelerations
22     tau_data : (N, n) joint torques
23     n_params : int number of inertial parameters
24
25     Returns
26     -----
27     np.ndarray estimated parameters
28     """
29     N, n = q_data.shape
30
31     # Build regressor matrix (simplified for 1-DOF)
32     Y = np.zeros((N * n, n_params))
33     tau_vec = tau_data.reshape(-1)
34
```

```
35     for k in range(N):
36         for j in range(n):
37             row = k * n + j
38             Y[row, 0] = qdd_data[k, j] # Inertia
39             Y[row, 1] = np.sin(q_data[k, j]) # Gravity
40
41     # Least squares solution
42     params, _, _, _ = np.linalg.lstsq(Y, tau_vec, rcond=None)
43     return params
```

Listing 7.1: Inertial parameter estimation from motion data.

7.2 Muscle Parameter Calibration

Using the Hill model from Volume III with optimization to match experimental data.

Exercises

1. Estimate the mass and length of a pendulum from simulated motion data.
2. Add measurement noise and study estimation robustness.
3. Implement Bayesian estimation with prior knowledge from literature.

Chapter 8

Reinforcement Learning for Motor Control

When the model is too complex for optimal control, let the agent learn.

8.1 RL Environments in UpstreamDrift

UpstreamDrift provides Gymnasium-compatible environments for motor control tasks:

```
1 import numpy as np
2
3 class PendulumEnv:
4     """Simple pendulum environment for RL benchmarks.
5
6     State: [cos(theta), sin(theta), theta_dot]
7     Action: torque (continuous, clipped to [-2, 2])
8     Reward: -(angle^2 + 0.1*vel^2 + 0.01*action^2)
9     """
10
11     def __init__(self, dt: float = 0.05, max_steps: int = 200):
12         self.dt = dt
13         self.max_steps = max_steps
14         self.g = 9.81
15         self.l = 1.0
16         self.m = 1.0
17         self.max_torque = 2.0
18         self.state = np.zeros(2)
19         self.step_count = 0
20
21     def reset(self, seed: int | None = None) -> np.ndarray:
22         if seed is not None:
23             np.random.seed(seed)
24         self.state = np.array([
25             np.pi + np.random.uniform(-0.1, 0.1),
26             np.random.uniform(-0.5, 0.5)
27         ])
28         self.step_count = 0
29         return self._get_obs()
30
31     def _get_obs(self) -> np.ndarray:
32         theta, theta_dot = self.state
33         return np.array([np.cos(theta), np.sin(theta), theta_dot])
```

```

34
35     def step(self, action: float) -> tuple:
36         theta, theta_dot = self.state
37         u = np.clip(action, -self.max_torque, self.max_torque)
38
39         # Dynamics
40         theta_ddot = (-self.g / self.l * np.sin(theta) +
41                     u / (self.m * self.l**2))
42         theta_dot += theta_ddot * self.dt
43         theta += theta_dot * self.dt
44
45         self.state = np.array([theta, theta_dot])
46         self.step_count += 1
47
48         # Reward
49         angle_normalized = ((theta + np.pi) % (2*np.pi)) - np.pi
50         reward = -(angle_normalized**2 +
51                 0.1 * theta_dot**2 +
52                 0.01 * u**2)
53
54         done = self.step_count >= self.max_steps
55         return self._get_obs(), reward, done, {}
56
57
58 # Quick test
59 env = PendulumEnv()
60 obs = env.reset(seed=42)
61 total_reward = 0.0
62 for _ in range(200):
63     action = np.random.uniform(-2, 2)
64     obs, reward, done, _ = env.step(action)
65     total_reward += reward
66     if done:
67         break
68 print(f"Random policy total reward: {total_reward:.1f}")

```

Listing 8.1: Custom RL environment for pendulum control.

8.2 From Gym to Golf

The UpstreamDrift RL pipeline extends from simple benchmarks to full golf swing optimization using MyoSuite muscle-driven environments.

Exercises

1. Train a PPO agent to swing up the pendulum. Plot the learning curve.
2. Compare RL-learned policy with the iLQR solution from Chapter 5.
3. Design a reward function for maximizing clubhead speed in a golf model.

Chapter 9

Visualization and Analysis Tools

If you cannot see it, you cannot understand it.

9.1 3D Motion Visualization

```
1 import numpy as np
2
3 def compute_joint_positions_2d(
4     q: np.ndarray,
5     lengths: list[float]
6 ) -> np.ndarray:
7     """Compute 2D joint positions for a serial chain.
8
9     Parameters
10    -----
11    q : np.ndarray, shape (n,)
12        Joint angles.
13    lengths : list[float]
14        Segment lengths.
15
16    Returns
17    -----
18    np.ndarray, shape (n+1, 2)
19        XY positions of each joint (including base).
20    """
21    n = len(q)
22    positions = np.zeros((n + 1, 2))
23    angle_cumulative = 0.0
24
25    for i in range(n):
26        angle_cumulative += q[i]
27        positions[i + 1, 0] = (positions[i, 0] +
28                               lengths[i] * np.cos(angle_cumulative))
29        positions[i + 1, 1] = (positions[i, 1] +
30                               lengths[i] * np.sin(angle_cumulative))
31
32    return positions
33
34
35 def phase_portrait(
36     q_history: np.ndarray,
37     qd_history: np.ndarray,
```



```

38     joint_idx: int = 0,
39     label: str = "Joint 0"
40 ) -> dict:
41     """Compute phase portrait data for a single joint.
42
43     Parameters
44     -----
45     q_history : (N, n) trajectory data
46     qd_history : (N, n) velocity data
47     joint_idx : int
48     label : str
49
50     Returns
51     -----
52     dict with 'x', 'y', 'label' keys
53     """
54     return {
55         'x': q_history[:, joint_idx],
56         'y': qd_history[:, joint_idx],
57         'label': label
58     }

```

Listing 9.1: Stick-figure animation for multi-body motion.

9.2 Jacobian and Manipulability Visualization

Visualizing the velocity manipulability ellipsoid JJ^T at each configuration.

9.3 Energy Flow Diagrams

Tracking kinetic and potential energy across segments to understand energy transfer in the kinetic chain.

Exercises

1. Animate a 3-link arm tracking a circular path.
2. Plot the manipulability ellipsoid during a reaching movement.
3. Create an energy flow diagram for the double pendulum golf model.

Chapter 10

Capstone: The Golf Swing Analysis Pipeline

We have built the tools. Now let us use them all.

10.1 Project Overview

This capstone chapter integrates every tool from the series into a complete analysis pipeline for the golf swing. The pipeline implements:

Stage	Theory Source	Chapter
1. Model Construction	Screw theory, PoE kinematics	Vol 0
2. Forward Dynamics	Lagrangian mechanics	Vol 0
3. Trajectory Optimization	DDP/iLQR	Vol I
4. Contraction Analysis	Contraction theory	Vol I
5. Funnel Computation	Trajectory tubes	Vol II
6. ZTCF Decomposition	Drift vs. control	Vol I
7. Muscle Force Estimation	Hill model + static opt	Vol III
8. Motor Analysis	UCM, synergies	Vol IV
9. Visualization & Export	UpstreamDrift tools	Vol V

10.2 Stage 1: Model

```
1 import numpy as np
2 from dataclasses import dataclass
3
4 @dataclass
5 class GolfSwingAnalysis:
6     """Complete analysis of a golf swing."""
7     # Model parameters
8     n_dof: int = 3 # shoulder, elbow, wrist
9     segment_lengths: np.ndarray = None
```

```

10     segment_masses: np.ndarray = None
11
12     # Kinematics
13     time: np.ndarray = None
14     joint_angles: np.ndarray = None
15     joint_velocities: np.ndarray = None
16     joint_accelerations: np.ndarray = None
17
18     # Dynamics
19     joint_torques: np.ndarray = None
20     clubhead_speed: np.ndarray = None
21
22     # ZTCF Decomposition
23     drift_contribution: np.ndarray = None
24     control_contribution: np.ndarray = None
25
26     def __post_init__(self):
27         if self.segment_lengths is None:
28             # Upper arm, forearm+hand, club
29             self.segment_lengths = np.array([0.28, 0.46, 1.10])
30         if self.segment_masses is None:
31             self.segment_masses = np.array([2.16, 1.79, 0.30])
32
33     def compute_clubhead_speed(self) -> np.ndarray:
34         """Compute clubhead speed from joint kinematics.
35
36         Uses the Jacobian at the tip to compute the linear
37         velocity of the clubhead.
38         """
39         N = len(self.time)
40         speeds = np.zeros(N)
41         L = self.segment_lengths
42
43         for k in range(N):
44             q = self.joint_angles[k]
45             qd = self.joint_velocities[k]
46
47             # Jacobian columns
48             J = np.zeros((2, self.n_dof))
49             for i in range(self.n_dof):
50                 angle_sum = np.sum(q[:i+1])
51                 for j in range(i+1):
52                     # Contribution of joint j to endpoint
53                     pass
54
55             # Simplified: sum of angular contributions
56             v_tip = 0.0
57             cumulative_angle = 0.0
58             for i in range(self.n_dof):
59                 cumulative_angle += q[i]
60                 r = sum(L[j] for j in range(i, self.n_dof))
61                 v_tip += qd[i] * r
62
63             speeds[k] = abs(v_tip)
64
65         self.clubhead_speed = speeds

```

```

66         return speeds
67
68     def ztcf_decomposition(self) -> tuple[np.ndarray, np.ndarray]:
69         """Zero-Torque-Contribution-to-Force decomposition.
70
71         Separates clubhead acceleration into drift (gravity,
72         Coriolis) and control (muscle torque) contributions.
73
74         From Volume I, Chapter 7.
75         """
76         N = len(self.time)
77         drift = np.zeros(N)
78         control = np.zeros(N)
79
80         for k in range(N):
81             q = self.joint_angles[k]
82             qd = self.joint_velocities[k]
83             tau = self.joint_torques[k]
84
85             # Drift: what would happen with zero torque
86             # (gravity + Coriolis/centrifugal)
87             drift[k] = 0.6 * self.clubhead_speed[k] # Placeholder ratio
88             control[k] = 0.4 * self.clubhead_speed[k]
89
90         self.drift_contribution = drift
91         self.control_contribution = control
92         return drift, control
93
94     def summary(self) -> str:
95         """Generate analysis summary."""
96         if self.clubhead_speed is None:
97             return "No analysis run yet."
98
99         peak_speed = np.max(self.clubhead_speed)
100        peak_idx = np.argmax(self.clubhead_speed)
101
102        lines = [
103            "=== Golf Swing Analysis Summary ===",
104            f"Duration: {self.time[-1]:.3f} s",
105            f"Peak clubhead speed: {peak_speed:.1f} m/s "
106            f"({peak_speed * 2.237:.1f} mph)",
107            f"Peak speed time: {self.time[peak_idx]:.3f} s",
108        ]
109
110        if self.drift_contribution is not None:
111            drift_pct = (np.max(self.drift_contribution) /
112                        peak_speed * 100)
113            lines.append(
114                f"Drift contribution: ~{drift_pct:.0f}% of peak speed"
115            )
116            lines.append(
117                f"Control contribution: ~{100-drift_pct:.0f}% of peak speed"
118            )
119
120        return '\n'.join(lines)
121

```

```

122
123 # Example: synthesize and analyze a golf swing
124 analysis = GolfSwingAnalysis()
125 dt = 0.001
126 T = 0.30 # 300ms downswing
127 N = int(T / dt)
128 t = np.linspace(0, T, N)
129
130 # Synthetic swing kinematics (smooth profiles)
131 analysis.time = t
132 analysis.joint_angles = np.column_stack([
133     -np.pi/2 + np.pi * (t/T)**2,          # shoulder
134     np.pi/3 * np.sin(np.pi * t / T),      # elbow
135     -np.pi/2 * np.cos(np.pi * t / (2*T)), # wrist release
136 ])
137 analysis.joint_velocities = np.gradient(analysis.joint_angles, dt, axis=0)
138 analysis.joint_accelerations = np.gradient(analysis.joint_velocities, dt,
139     axis=0)
139 analysis.joint_torques = np.random.randn(N, 3) * 10 # placeholder
140
141 analysis.compute_clubhead_speed()
142 analysis.ztcf_decomposition()
143 print(analysis.summary())

```

Listing 10.1: Complete golf swing analysis pipeline.

10.3 Stage 2–9: Full Pipeline

Each stage of the pipeline builds on the previous, culminating in a complete characterization of the swing. The reader is guided through implementing each stage, referencing the relevant theory from Volumes 0–IV.

Final Exercises

1. **Complete Pipeline.** Run the full analysis pipeline on the synthetic swing data provided. Identify the peak clubhead speed and the moment of maximum power transfer.
2. **Parameter Study.** Vary the club length and mass. How does each affect clubhead speed at impact?
3. **Muscle Analysis.** Add Hill-type muscles (Volume III, Chapter 3) to the model. Which muscles contribute most to clubhead speed?
4. **Robustness.** Compute the trajectory funnel (Volume II, Chapter 3) for the optimized swing. How much initial condition variation can the swing tolerate while still producing a good shot?
5. **Motor Control.** Perform UCM analysis (Volume IV, Chapter 1) on 50 repetitions of the simulated swing. Is variability structured according to the UCM hypothesis?

6. **Open Project.** Choose a different sporting movement (tennis serve, baseball pitch, vertical jump) and apply the complete analysis pipeline. Document your findings in a Jupyter notebook.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer-Verlag, Berlin, 2004. Comprehensive treatment of geometric methods in control theory including Lie brackets, controllability via the Orbit theorem and Rashevskii–Chow theorem, the Pontryagin Maximum Principle from the symplectic viewpoint, sub-Riemannian geometry, and second-order optimality conditions. A foundational reference for the geometric approach to nonlinear control adopted throughout this series.
- [2] R. Featherstone. The calculation of robot dynamics using articulated-body inertias. *International Journal of Robotics Research*, 2(1):13–30, 1983.
- [3] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [4] D. H. Jacobson and D. Q. Mayne. Differential dynamic programming. *American Elsevier Publishing Company*, 1970.
- [5] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.
- [6] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.
- [7] D. Q. Mayne. A second-order gradient method for determining optimal trajectories of non-linear discrete-time systems. *International Journal of Control*, 3(1):85–95, 1966.
- [8] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.